



# United International University

## Department of Computer Science and Engineering

Object Oriented Programming Laboratory

Topic: Inheritance, Encapsulation, & Constructor Chaining

Duration: 2.5

hours

### Lab Sheet: Object-Oriented Programming – Inheritance & Encapsulation

---

**Instructional Guide:** Prepared by Ashraful Islam Paran, Dept. of CSE, UIU. Complete the programming tasks within your designated lab window. Ensure proper structural access modification across all custom types.

## 1. Objectives

By the end of this practical framework, students will be able to:

- Implement Single, Multilevel, and Hierarchical inheritance relationships cleanly in Java.
- Explain the design decisions preventing multiple class inheritance within the JVM architecture.
- Apply object encapsulation protocols using private tracking fields matched with public access channels.
- Manage object instantiation tracks using explicit `super()` and `this()` chaining references.
- Override instance actions and reference base implementations accurately via `super`.

## 2. Core Principles: Inheritance

Inheritance allows a structured subclass to acquire the behaviors and internal state templates of a parent superclass through the `extends` keyword.

### Core Advantages:

- **Code Reusability:** Common data attributes and functional behaviors are structured in a central repository.
- **Method Overriding:** Subclasses can provide distinct localized logic for general parent operations.
- **Polymorphic Interoperability:** Parent reference points can hold child instances interchangeably at execution time.

## 3. Supported Inheritance Typologies

- **Single Inheritance:** A standalone type explicitly targets one base definition (`class B extends A`).
- **Multilevel Inheritance:** A multi-tiered structural sequence (`class C extends B` where `class B extends A`).
- **Hierarchical Inheritance:** Multiple isolated configurations branch from a shared base (`B, C, D extend A`).

- **Multiple & Hybrid Models:** Blocked for standard class schemas to resolve execution ambiguities; implemented only via interfaces.

## 4. Example 1: Single Inheritance Patterns

The program below illustrates a basic structural extension relationship where sub-entities consume general parent execution paths.

SingleInheritanceDemo.java

```
class Animal {
    void eat() {
        System.out.println("This animal eats food.");
    }
}

class Dog extends Animal {
    void bark() {
        System.out.println("The dog barks.");
    }
}

public class SingleInheritanceDemo {
    public static void main(String[] args) {
        Dog d = new Dog();
        d.eat(); // Inherited from parent reference tracking
        d.bark(); // Local custom subclass configuration
    }
}
```

This animal eats food. The dog barks.

### Task 4.1 – Single Inheritance Practice (10 min)

- (a) Define an independent class called **Bird** that branches off from the **Animal** base block.
- (b) Add a dedicated, isolated execution behavior named **fly()** inside the **Bird** configuration block.
- (c) Within an active workspace, initialize a **Bird** reference, invoking both the shared **eat()** and localized **fly()** channels.
- (d) Test polymorphism principles by assigning an explicit child instantiation reference directly into a parent **Animal** variable layout.

## 5. Example 2: Multilevel Inheritance Operations

This example profiles generational tracking chains, allowing deep leaf types to invoke ancestral attributes upstream.

MultilevelInheritanceDemo.java

```
class Animal {
    void eat() {
        System.out.println("This animal eats food.");
    }
}
```

```

}

class Dog extends Animal {
    void bark() {
        System.out.println("The dog barks.");
    }
}

class Puppy extends Dog {
    void weep() {
        System.out.println("The puppy weeps.");
    }
}

public class MultilevelInheritanceDemo {
    public static void main(String[] args) {
        Puppy p = new Puppy();
        p.eat(); // Roots from the foundational parent block
        p.bark(); // Extracted from the intermediate component block
        p.weep(); // Evaluates local object tracking metrics
    }
}

```

This animal eats food. The dog barks. The puppy weeps.

### Task 5.1 – Multilevel Track Implementations (12 min)

Construct a concrete three-tiered structural hierarchy mapping out: `Vehicle` → `Car` → `SportsCar`.

- Provide the `Vehicle` type with base lifecycle hooks named `start()` and `stop()`.
- Expand the structural definition within `Car` by attaching an accelerated velocity method named `accelerate()`.
- Furnish `SportsCar` with a distinct localized behavior loop termed `nitroBoost()`.
- Instantiate a concrete `SportsCar` reference block inside a test method workspace, proving access coverage across all four structural actions.

## 6. Example 3: Hierarchical Inheritance Environments

Hierarchical configurations allow multiple sub-templates to derive properties from a single generalized core pattern.

HierarchicalInheritanceDemo.java

```
class Animal {
    void eat() {
        System.out.println("This animal eats food.");
    }
}

class Dog extends Animal {
    void bark() { System.out.println("The dog barks."); }
}

class Cat extends Animal {
    void meow() { System.out.println("The cat meows."); }
}

class Puppy extends Dog {
    void weep() { System.out.println("The puppy weeps."); }
}

public class HierarchicalInheritanceDemo {
    public static void main(String[] args) {
        Dog d = new Dog();
        Cat c = new Cat();
        Puppy p = new Puppy();

        d.eat(); d.bark();
        c.eat(); c.meow();
        p.eat(); p.bark(); p.weep();
    }
}
```

This animal eats food. The dog barks. This animal eats food. The cat meows.  
This animal eats food. The dog barks. The puppy weeps.

### Task 6.1 – Hierarchical Architecture Exercises (15 min)

- (a) Establish a clean structural parent model named **Shape** tracking a base execution pathway called **draw()**.
- (b) Implement three separate isolated child variants: **Circle**, **Rectangle**, and **Triangle**, each overriding the baseline **draw()** message stream.
- (c) Within your main workspace loop, isolate separate instances of each model variant to show individual terminal outputs.

## 7. Example 4.1: Data Encapsulation – Managed Student Models

Encapsulation isolates structural states using selective access barriers. This protects internal tracking variables from direct, unverified outer modifications.

EncapsulatedStudentDemo.java

```
class Student {
    // Structural parameters are locked via the private access identifier
    private String name;
    private int id;

    // Rigid initialization controller constructor
    public Student(String name, int id) {
        this.name = name;
        this.id = id;
    }

    // Public retrieval methods act as formal information pathways
    public String getName() {
        return name;
    }

    public int getId() {
        return id;
    }

    public void showInfo() {
        System.out.println("Name: " + name + " | ID: " + id);
    }
}

public class EncapsulatedStudentDemo {
    public static void main(String[] args) {
        Student s = new Student("Rahim", 2219001);
        s.showInfo();

        // Direct adjustments below break compiler validation protocols (Good Design)
        // s.name = "Karim";
        // System.out.println(s.id);

        System.out.println("Name from getter: " + s.getName());
    }
}
```

Name: Rahim | ID: 2219001 Name from getter: Rahim

## 8. Example 4.2: Combining Encapsulation with Constructor Chains

This example demonstrates structural initialization routing across matching configurations using the keywords `this()` and `super()`.

ChainedEncapsulationDemo.java

```
class Animal {
    private String name;

    Animal() {
        this("Unknown Animal"); // Forwards execution track to parameter
                                // constructor
    }

    Animal(String name) {
        this.name = name;
    }

    void eat() {
        System.out.println(name + " is eating.");
    }

    public String getName() { return name; }
}

class Dog extends Animal {
    Dog() {
        super(); // Triggers the default ancestral block routing
    }

    Dog(String name) {
        super(name); // Maps straight to custom parameter channels upstream
    }

    void bark() {
        System.out.println(getName() + " is barking.");
    }
}

public class ChainedEncapsulationDemo {
    public static void main(String[] args) {
        Dog d1 = new Dog();
        Dog d2 = new Dog("Buddy");
        d1.eat(); d1.bark();
        d2.eat(); d2.bark();
    }
}
```

Unknown Animal is eating. Unknown Animal is barking. Buddy is eating. Buddy is barking.

### Task 7.1 – Architectural Encapsulation & Chaining (20 min)

- (a) Draft an encapsulated data structure named **Student** containing strict private variables for tracking: **name**, **id**, and **cgpa**.
- (b) Formulate three initialization pathways within the type definition:
  - A default parameterless constructor that routes default data flags forward via: `this("Unknown", 000, 0.0)`.
  - A single-argument constructor processing identity strings exclusively.
  - A comprehensive three-argument parameter constructor that handles complete structural data fields.
- (c) Apply inline chaining via `this()` to reduce redundant initialization paths within the class template.
- (d) Expose reading metrics across all variables using public **getter** pathways (omit modifying setter actions for this drill).
- (e) Initialize independent test items tracking all three constructor configurations inside an active executable project block.

## 9. Example 5: Overriding Logic and Parent Element Invocations

The sample code below highlights polymorphism techniques where child types re-engineer common actions, using the **super** keyword to access base rules.

OverridingInvocationDemo.java

```
class Animal {
    void sound() {
        System.out.println("The animal makes a sound");
    }
}

class Dog extends Animal {
    @Override
    void sound() {
        super.sound(); // Redirects execution flow temporarily back to base
        rules
        System.out.println("The dog says: Bow Wow");
    }
}

public class OverridingInvocationDemo {
    public static void main(String[] args) {
        Dog d = new Dog();
        d.sound();
    }
}
```

The animal makes a sound The dog says: Bow Wow

### Task 8.1 – Strategic Method Overriding Practice (12 min)

Construct a flexible organizational workflow modeling framework matching these requirements:

- Define an `Employee` type running a task function called `work()` which prints: "Employee works".
- Formulate a derived type called `Manager` extending `Employee`. Override the `work()` action loop so that it calls `super.work()` before printing: "Manager manages team".
- Extend the structural workspace models by mapping a separate `Developer` sibling branch that overrides `work()` to output: "Developer writes code".

## 10. Real-World Paradigm: Enterprise Payroll Architectures

This end-to-end framework profiles object context inheritance combined with access constraints. It demonstrates constructor initialization tracks routing upward through explicit parent fields.

EnterpriseSystemDemo.java

```
class Employee {
    private String name;
    private double salary;

    Employee(String name, double salary) {
        this.name = name;
        this.salary = salary;
    }

    void display() {
        System.out.println("Name: " + name + ", Salary: $" + salary);
    }

    public double getSalary() { return salary; }
}

class Manager extends Employee {
    private double bonus;

    Manager(String name, double salary, double bonus) {
        super(name, salary); // Explicitly maps parameter states upward
        this.bonus = bonus;
    }

    @Override
    void display() {
        System.out.println("Manager: " + super.getClass().getSimpleName() +
            " | Total Pay: $" + (getSalary() + bonus));
    }
}

public class EnterpriseSystemDemo {
    public static void main(String[] args) {
        Manager m = new Manager("Karim", 60000, 20000);
        m.display();
    }
}
```

Manager: Manager | Total Pay: 80000.0

## 11. Example 10: Isolating Variable Scopes via super

This structural outline demonstrates how to explicitly bypass localized variable shadows inside multi-tiered subclass definitions using parent field markers.

ScopeIsolationDemo.java

```
class Parent {
    int x = 10;
}

class Child extends Parent {
    int x = 20;

    void display() {
        System.out.println("Child x      = " + x);      // Resolves local
        // frame value
        System.out.println("Child this.x = " + this.x); // Targets explicitly
        // local coordinates
        System.out.println("Parent x     = " + super.x); // Bypasses local
        // shadow to hit base state
    }
}

public class ScopeIsolationDemo {
    public static void main(String[] args) {
        new Child().display();
    }
}
```

Child x = 20 Child this.x = 20 Parent x = 10

## 12. Example 11: Explicit Parent Functional Triggers

This template maps execution steps through nested tracking methods, calling parent operations before executing specialized localized processes.

ParentMethodTriggerDemo.java

```
class Parent {
    void greet() {
        System.out.println("Hello from Parent");
    }
}

class Child extends Parent {
    @Override
    void greet() {
        super.greet(); // Dispatches execution tracking line into ancestral
        logic
        System.out.println("Hello from Child");
    }
}

public class ParentMethodTriggerDemo {
    public static void main(String[] args) {
        Child c = new Child();
        c.greet();
    }
}
```

Hello from Parent Hello from Child

## 13. Example 12: Base Parameterless Constructor Tracing

This block tracks implicit constructor chains during subclass allocation. It shows how parent types configure their foundations before child setups run.

DefaultChainTracingDemo.java

```
class Parent {
    Parent() {
        System.out.println("Parent constructor called");
    }
}

class Child extends Parent {
    Child() {
        super(); // Explicit statement insertion (optional if no-arg remains
        default)
        System.out.println("Child constructor called");
    }
}
```

```

public class DefaultChainTracingDemo {
    public static void main(String[] args) {
        new Child();
    }
}

```

Parent constructor called Child constructor called

## 14. Example 13: Passing Parameters up the Constructor Chain

This example shows how structured subclass containers use target variables to initialize required states within ancestral definitions.

ParameterizedChainDemo.java

```

class Person {
    String name;
    Person(String name) {
        this.name = name;
        System.out.println("Person constructor: " + name);
    }
}

class Student extends Person {
    int id;
    Student(String name, int id) {
        super(name); // Maps parameters up to base state configurations
        this.id = id;
        System.out.println("Student constructor: ID = " + id);
    }
}

public class ParameterizedChainDemo {
    public static void main(String[] args) {
        Student s = new Student("Ashraful", 2219022);
    }
}

```

## 15. Example 14: Complex Inter-Class Constructor Chaining

This end-to-end framework visualizes nested object initialization sequencing across structural class tiers using both `this()` and `super()` tracking tags.

ComplexChainDemo.java

```
class Person {
    String name;
    int age;

    Person() {
        this("Unknown", 0);
        System.out.println("Person() called");
    }

    Person(String name) {
        this(name, 0);
        System.out.println("Person(String) called");
    }

    Person(String name, int age) {
        this.name = name;
        this.age = age;
        System.out.println("Person(String, int) called");
    }
}

class Student extends Person {
    int roll;

    Student() {
        super();
        System.out.println("Student() called");
    }

    Student(String name, int age, int roll) {
        super(name, age);
        this.roll = roll;
        System.out.println("Student(full) called");
    }
}

public class ComplexChainDemo {
    public static void main(String[] args) {
        System.out.println("--- Creating Student() ---");
        Student s1 = new Student();

        System.out.println("\n--- Creating Student(full) ---");
    }
}
```

```
        Student s2 = new Student("Alice", 20, 101);
    }
}
```

```
-- Creating Student() -- Person(String, int) called Person(String) called
Person() called Student() called
-- Creating Student(full) -- Person(String, int) called Student(full) called
```

## 16. Structural Synthesis Blueprint Summary

The table below details the execution characteristics and syntax parameters for core Java inheritance and encapsulation structures.

| OOP Mechanism               | Architectural Operational Scope                             | Syntax Prototype                      |
|-----------------------------|-------------------------------------------------------------|---------------------------------------|
| Single Extension            | Forms a straightforward subclass link to one parent.        | <code>Dog extends Animal</code>       |
| Multilevel Link             | Constructs an extended structural ancestor hierarchy.       | <code>Puppy extends Dog</code>        |
| Hierarchical Model          | Branches multiple separate child types off a common core.   | <code>Cat, Dog extend Animal</code>   |
| Encapsulation               | Locks field mutations behind private access boundaries.     | <code>private String identity;</code> |
| <code>super.variable</code> | References shadowed state fields inside base class blocks.  | <code>super.targetMetric;</code>      |
| <code>super.method()</code> | Invokes parent operational logic from an overridden method. | <code>super.executeAction();</code>   |
| <code>super()</code>        | Directs constructor initialization upward to parent types.  | <code>super(name, age);</code>        |
| <code>this()</code>         | Routes tracking to alternate local constructor mappings.    | <code>this("Fallback", 0);</code>     |
| Method Overriding           | Re-engineers an inherited action with localized logic.      | <code>@Override void process()</code> |

## 17. Practical System Design Challenge

Design a modular vehicle asset pipeline using structured types that strictly maintain data isolation and type inheritance.

- **\*\*Base Component\*\***: `Vehicle`
- **\*\*Derived Structural Layouts\*\***: `Car` and `Bike` branching hierarchically from the base layout.
- **\*\*Leaf Target Component\*\***: `ElectricCar` deriving directly from the intermediate `Car` infrastructure.

Ensure every type isolates its variables with explicit visibility filters, maintains clean constructor forwarding tracks, runs secure get/set validation layers, and overrides baseline reporting operations.

## 18. Analytical Evaluation Review Questions

1. Explain how Java resolves structural diamond path conflicts by restricting multiple class extensions at the compilation tier.

2. Contrast the operational usage profiles of the keyword instructions **super** versus explicit structural **super()** method invocations.
3. Sketch the structural template definitions required to map a clean hierarchical layout using the blueprint sequence: **Vehicle**  $\rightarrow$  {**Car**, **Bike**, **Truck**}.
4. Trace out the construction stack calls running through a multi-tiered context sequence matching the path: **Person**  $\rightarrow$  **Student**  $\rightarrow$  **GraduateStudent**, utilizing optimized constructor chains.
5. Build a banking asset model tracing a **BankAccount** framework down to a specialized **SavingsAccount** configuration. Override calculation functions to evaluate custom monthly interest logic.
6. Analyze the compilation behavior that occurs when an operational instruction loop is placed before a **super()** or **this()** constructor chaining call.

### **Keep Coding!**

*Mastering inheritance hierarchies and structural access constraints builds a resilient foundation for enterprise software engineering.*